GEMM 笔记：代码版本演进总结 与 自我检验清单详解

> 本文前半部分总结本目录 `base.cu → V7.cu` 的代码版本演进与分析，
> 后半部分逐条回答 AIInfraGuide 模块二 4.1《CUDA GEMM 算子性能优化》的「自我检验清单」。
> 所有数字取自各 `.cu` 文件头部记录的 benchmark。
> 数字分两批：**RTX 2080 Ti**（原文实测）与 **RTX 5070 Ti**（本仓库当前 `.cu` 头注释）；两卡逐版对比见第一部分末尾「RTX 2080 Ti 与 RTX 5070 Ti 实测对比」。

第一部分：GEMM 目录代码版本总结与分析

版本总览（RTX 2080 Ti 实测）

> 注意：本表是 **RTX 2080 Ti** 的记录，其中**只有 `base.cu` 是 1024³**，V1 之后都是 2048³，
> 所以 base 不能和后面对比；不同 GPU / 编译选项下的数字也不可直接横比。
> （5070 Ti 上 `base.cu` 已按 2048³ 重测，可直接参与对比，见第一部分末尾对比表。）

版本	记录配置	时间 (ms)	TFLOPS	核心技术	相对前者
`base.cu`	1024³	2.357	0.911	朴素内积，每线程一个输出	基线
`V1.cu`	2048³	2.767	6.208	Block Tiling（BM=BN=128, BK=8, 256 线程，每线程 8×8）	大幅提升
`V2.cu`	2048³	3.975	4.322	线程级寄存器外积（TM×TN=8×8）	**反而变慢**
`V2F.cu`	1024³ / 2048³	0.544 / 2.656	3.948 / 6.468	V2 + float4（Global→Shared、Shared→Reg）	比 V2 快，接近 V1
`V3.cu`	2048³	5.362	3.204	Warp Tiling（block 内 4×2 warp，warp tile 32×64，lane	**更慢**

版本	记录配置	时间 (ms)	TFLOPS	核心技术	相对前者
				4×8）	
`V3F.cu`	2048³	2.634	6.523	V3 + float4 加载	明显回升
`V4.cu`	2048³	2.005	8.569	Scatter 4×4 + A 转置 + float4	大幅提升
`V4P.cu`	2048³	1.993	8.621	V4 + Shared Memory Padding（+4）	小幅提升
`V4Z.cu`	2048³	1.953	8.796	V4P + Z-order lane 映射	小幅提升
`V5.cu`	2048³	1.777	9.668	软件双缓冲（Ping-Pong）	提升
`V51.cu`	2048³	1.776	9.675	V5 微调（迭代次数变量化），本质同 V5	持平
`V6.cu`	无记录	—	—	cp.async + STAGES 级流水线（sm_80+）	目标更快，未实测

配套 profiler 文件（原文所列）：`V2F_report.ncu-rep`、`V3_report.ncu-rep`、`V3F_report.ncu-rep`、`V3F_full.ncu-rep`、`V4_report.ncu-rep`、`v4z_report.ncu-rep`、`v51_report.ncu-rep`。**注：这些 `*.ncu-rep` 在当前目录中已不存在**（目录内只有 `.cu` / `.exe`、`note.md`、`note1.md`、`GEMM.pdf`），此处仅保留文件名作索引。

演进主线：从"减少搬运"到"重叠搬运"

整个目录走的是 GEMM 优化的标准三段式：

1. **减少 Global 流量**：用 Block Tiling 把 A/B 的 tile 搬进 Shared Memory，让 block 内所有输出复用（AI 0.25 → 32）；
2. **减少 Shared 流量**：把复用再下沉到寄存器，用外积把 `LDS:FMA` 从 2:1 压到约 1:4；
3. **隐藏访存延迟**：当搬运指令数降下来后，用双缓冲 / `cp.async` 让"搬下一块"和"算这一块"重叠。

但本目录真实的数据说明：**手段不等于收益**，中间几版反而更慢，必须靠 profiler 和实测来定位瓶颈。

逐版本分析

阶段一：基线 `base.cu`

- `Block = 16×16` ， `Grid = 64×64` ，每个线程独立算一个 `C[row][col]` ，沿 K 做内积。
- 没有任何显式复用，A 的一行、B 的一列被大量线程重复从 Global 读。
- AI = 0.25 FLOPs/Byte，**瓶颈是 Global Memory 带宽/延迟**（Long Scoreboard）。
- 1024^3 只有 0.911 TFLOPS，几乎完全被带宽拖住 —— 这正是要分块的原因。

阶段二：Block Tiling —— `V1.cu`

- `BM=BN=128, BK=8, 256 threads` ；每线程负责 8×8 输出（ `Tm=Tn=8` ）。
- 协作加载：256 线程分别按 32×8 / 8×32 的线程重排把 `As[BM][BK]` 、 `Bs[BK][BN]` 搬进 shared，再 `__syncthreads()` 。
- 内层用**外积**：每 k 步读 TM+TN 个 shared 值，做 TM·TN 次 FMA； `LDS:FMA ≈ 1:4` 。
- 效果：0.911 → 6.208 TFLOPS（同为不同尺寸，仅作趋势参考），AI 达到理论 32。
- 瓶颈迁移：从 Global 带宽转移到 **Shared 带宽 / LSU(MIO)** 以及加载指令的模式。

阶段三：线程级复用 —— `V2.cu` / `V2F.cu`

- `V2.cu` 把"每线程 TM×TN 寄存器累加器 + 显式外积"独立出来： `LDS:FMA` 更优，理论上 shared 更省。
- 但实测 **3.975 ms / 4.322 TFLOPS，比 V1 慢**。原因：显式寄存器分块引入了更多寄存器压力与更复杂的加载/索引，且线程到数据的映射没有配套优化 —— **"复用下沉"本身不保证更快**。
- `V2F.cu` 在 V2 上加 **float4 向量化**（Global→Shared 与 Shared→Reg 都向量化）：
  - 1024^3：0.544 ms / 3.948 TFLOPS；
  - 2048^3：2.656 ms / 6.468 TFLOPS（追平、略超 V1）。
  - 说明真正见效的是 **访存指令宽度**（LDG.128 / LDS.128），把 4 次标量 load 合并成 1 次。

阶段四：Warp 映射 —— `V3.cu` / `V3F.cu`

- `V3.cu` 显式划分 Warp tile：block 内 8 个 warp 排成 4×2，每个 warp 覆盖 `32×64` ，warp 内 lane 排成 4×8。
- 目的是让线程→输出 tile 的归属更清楚，为后续 broadcast / bank 优化打基础。
- 但 `V3.cu` 实测 **5.362 ms / 3.204 TFLOPS，反而最慢**：映射本身没错，但此时**加载仍是标量、shared 访问没有优化**，映射带来的开销没被收益抵消。
- `V3F.cu` （V3 + float4）回到 **2.634 ms / 6.523 TFLOPS** —— 再次印证真正救场的是向量化访存，而非映射形式。

- 教训：**warp/lane 布局只有在访存模式和 bank 行为配套优化后才有意义**。

阶段五：访存布局 —— `V4.cu` / `V4P.cu` / `V4Z.cu`（本目录的关键跃升）

这是提升最大的一组，从 ~6.5 → ~8.8 TFLOPS。

- `V4.cu`（8.569 TFLOPS）：三件事组合：
  i. **A 转置存放** `As[BK][BM]`，使后续 M 方向数据连续；
  ii. `float4` **加载/读取** A、B，减少访存指令数；
  iii. **Scatter 4×4** 布局：每线程负责 4 个分散的 4×4 子块，重新安排线程读取与输出片段。
  （这是多个布局改动的组合实验，收益不能单独归给某一项。）
- `V4P.cu`（8.621 TFLOPS）：加 **Shared Padding** —— 行 stride 从 `BM(128)` 补到 `BM+4(132)`。`128 % 32 == 0` 会导致转置写入 `As[col+i][row]` 时不同 k 落同一个 bank；`132 % 32 == 4` 正好错开。补 4（而非 1）是为了**保持 16B 对齐**，让 float4 仍可用；同时把 B 的 4 次标量写合并成 1 次 float4 写。
- `V4Z.cu`（8.796 TFLOPS）：在 V4P 上把 **quarter-warp 的 lane 映射从 1×8 行主序改成 2×4 行交错（Z-order）**。
  用 ncu 实测（每条 shared load 指令的 L1 wavefront 数）：tileA `2.00 → 2.00`（本就是广播，降不动），tileB `4.00 → 2.00`，合计 `3.00 → 2.00`，每 k 步从 12 → 8 个。**这是用 profiler 数据驱动的微优化**，收益约 0.2 TFLOPS。
- 小结：这一阶段的收益来自**把访存从"标量、易冲突"变成"向量化、错开 bank"**，而 Z-order 是在此基础上榨取的最后一点 shared 带宽。
- **注意（5070 Ti）**：这一步的"跃升"只在 2080 Ti 上成立。新卡上 `float4` 的收益已被 `V2F` 提前吃掉（V2F 20.41 ≈ V4 20.39），`V4` 相对 `V3F` 只剩约 +2%；后半程的增量主要来自 padding / Z-order 与流水线，而不是访存指令宽度。详见第一部分末尾对比。

阶段六：流水线 —— `V5.cu` / `V51.cu` / `V6.cu`

- `V5.cu`（9.668 TFLOPS）：在 V4Z 上加 **软件双缓冲（Ping-Pong）**。
  预取下一个 k-tile 到寄存器 `a_reg/b_reg`，同时用当前 shared 做外积；FFMA 那一大段计算正好盖住 Global load 的延迟，算完再把预取数据 STS 进另一块 buffer，最后 `__syncthreads()`。
  代价：要一直压着预取寄存器，必须 `__launch_bounds__(256, 2)` 控寄存器，否则 occupancy 掉到 25%。
- `V51.cu`（9.675 TFLOPS）：与 V5 本质相同，只是把加载迭代次数写成 `A_ITERS/B_ITERS` 变量（BK=8 时都是 1），性能持平，属于整理版本。
- `V6.cu`（**无 benchmark 记录**）：面向 Ampere（sm_80+）把软件双缓冲换成 `cp.async` + `STAGES=4` **多级流水线**。

- Global→Shared 不再经过寄存器，寄存器压力骤降；在飞 tile 从 1 个变成 `STAGES-1` 个，延迟容错更大；
- 三个坑： `cp.async` 每次只搬连续 4/8/16B（A 转置后只能 4B `.ca` ，B 可 16B `.cg` ）； `wait_group` 只保证本线程的拷贝，等完仍需 `__syncthreads()` ；尾部不足时也要 commit 空组，否则 `wait_group` 会误判。
- **因为没有实测数据，只能总结实现意图，不能断言它一定快于 V5**；在 sm_75 上它会退化为同步拷贝，反而更慢。
- **补充（5070 Ti 实测）**：新卡上 V6 已测得 **0.753 ms / 22.807 TFLOPS**，反而慢于 V5（0.680 ms / 25.252 TFLOPS）——这一版 `cp.async` （LDGSTS）+ STAGES=4 在 sm_120 上并不划算，新架构更适合 TMA / `cp.async.bulk` 。

关键结论

1. **优化层数 ≠ 性能**：V2、V3 增加了"更高级"的 tiling，实测却比 V1 慢。收益取决于访存布局、生成的指令、寄存器/shared 占用和硬件调度。
2. **本目录真正的大头收益来自三件事**：
   - Block tiling 提高 Global 复用（ `base → V1` ）；
   - `float4` 向量化 + shared 转置/padding（ `V3F → V4/V4P` ，~6.5 → ~8.6 TFLOPS）；
   - 流水线隐藏延迟（ `V4Z → V5` ，~8.8 → ~9.7 TFLOPS）。
3. **profiler 驱动**：V4Z 的 Z-order、V5/V6 的寄存器/occupancy 取舍，都是先用 ncu 看到瓶颈（bank/wavefront、Long Scoreboard、occupancy）再改的。
4. **配置要统一**： `base` 是 1024^3，其余是 2048^3，二者数字不可直接比较；跨 GPU、跨编译选项同样不可横比。
5. **正确性始终要验**：所有记录版本头部都标了 `Check = PASS` ；改动矩阵尺寸、尾块边界或对齐条件后必须重测。
6. **对齐/边界隐患**：V4 之后的 `float4` 、A 转置与 `cp.async` 路径都依赖地址 16B 对齐和 K/BM/BN 是 4 的倍数，尾块必须走标量兜底。
7. **换到 5070 Ti 后结论基本成立，但重心变了**：整体加速 ~2.4–3.2x，明显高于 1.45x 的显存带宽比，说明 2048^3 早已不受显存带宽限制；而 `V3F → V4` 的"关键跃升"在新卡上几乎消失（约 +2%），后半程收益转向 shared bank 冲突与延迟隐藏。

> 目录里还有一个名为 `-Xptxas=-v` 的文件，其实是编译命令写错位置产生的可执行文件（ `nvcc -Xptxas=-v ... -o` 的误用），不是源码，可忽略/删除。
> `V2F_report.ncu-rep` 等 `*.ncu-rep` 当时是 Nsight Compute 的采集结果；这些文件现已不在目录中（见前文「版本总览」注），如需复盘请重新采集。

RTX 2080 Ti 与 RTX 5070 Ti 实测对比

同一套代码（`base.cu` → `V7.cu`），分别在两代显卡上实测。两卡都统一取 **2048³**（5070 Ti 上 `base.cu` 已按 2048³ 重测；2080 Ti 的 base 记录是 1024³，仅作趋势参考）：

版本	2080 Ti 时间 (ms)	2080 Ti (TFLOPS)	5070 Ti 时间 (ms)	5070 Ti (TFLOPS)	加速比
`base.cu`	2.357 *(1024³)*	0.911	5.772	2.976	— *(尺寸不同)*
`V1.cu`	2.767	6.208	1.459	11.774	1.90x
`V2.cu`	3.975	4.322	1.555	11.049	2.56x
`V2F.cu`	2.656	6.468	0.842	20.411	3.16x
`V3.cu`	5.362	3.204	1.954	8.790	2.74x
`V3F.cu`	2.634	6.523	0.860	19.979	3.06x
`V4.cu`	2.005	8.569	0.842	20.394	2.38x
`V4P.cu`	1.993	8.621	0.818	20.991	2.43x
`V4Z.cu`	1.953	8.796	0.746	23.033	2.62x
`V5.cu`	1.777	9.668	0.680	25.252	2.61x
`V51.cu`	1.776	9.675	0.680 / 0.630	25.257 / **27.273**	2.61x / 2.82x
`V6.cu`	—	—	0.753	22.807	—
`V7.cu`	—	—	40.860 *(8192³)*	26.909	—

> 补充：`V5.cu` 另有 K=8192 记录（41.728 ms / 26.350 TFLOPS）；`V51.cu` 的 27.273 TFLOPS 来自 BK=16；`V7.cu` 为本次实跑（8192³，26.909 TFLOPS）。
> 5070 Ti 数据取自各 `.cu` 头注释（`V7` 为实跑补充）；2080 Ti 数据取自本文原表。

结论

1. **整体加速 ~2.4–3.2x（中位 ~2.6x），明显高于显存带宽比。** 两卡带宽 616 → 896 GB/s 只有 **1.45x**，而 FP32 峰值约 13.4 → ~44 TFLOPS 是 **~3.3x**。实测加速远高于 1.45x，说明 2048³ 的

GEMM 早已不是显存带宽瓶颈，收益主要来自片上（FFMA / shared）吞吐，所以新卡拿到的比"纯带宽"多得多。

2. **两卡的最优值都还没摸到 FP32 峰值。** 2080 Ti 最好 9.675 TFLOPS ≈ 72% 峰值；5070 Ti 最好 27.273 TFLOPS ≈ 62%。新卡相对峰值的利用率反而更低 —— 瓶颈已经移到片上数据搬运（shared bank / LDS 吞吐 / 延迟隐藏），而不是 FP32 算力。
3. **优化收益的"分布"变了。**
   - 2080 Ti： `V3F → V4` 是最大跃升（6.52 → 8.57，**+31%**），即 float4 + A 转置 + scatter。
   - 5070 Ti： `float4` 的收益提前被 `V2F` 吃掉（V2 11.05 → V2F 20.41）， `V2F → V3F → V4` 基本贴平（20.41 / 19.98 / 20.39）——**V4 那套组合改动在新卡上几乎不再加分**。
   - 反而 `V4P` （+3%）、 `V4Z` （+10%）、 `V5` （+10%）在后半程继续涨，说明新卡剩下的瓶颈更集中在 **shared bank 冲突与延迟隐藏**，而非访存指令宽度。
4. **stage-5"关键跃升"的地位下降。** 原文把 V4 称作"本目录的关键跃升"，这在 2080 Ti 上成立（+31%），在 5070 Ti 上只剩约 +2%。新卡的真正大头是 `base → V1` （2.98 → 11.77，Block Tiling 打满 Global 复用）和 `V1/V2 → V2F` （→ 20.41，float4 向量化）。
5. **`V6.cu` （cp.async）在 5070 Ti 上反而更慢**（22.807 vs `V5` 的 25.252）。 `sm_120` 本身支持 `cp.async` ，但 LDGSTS 的 4/8/16B 在这个尺寸下本就是过渡方案，且 BK=8 + STAGES=4 会抬高 shared 占用、压低 occupancy；新卡上更合适的是 TMA / `cp.async.bulk` （ `sm_90+` ），本目录没有采用。
6. **`V7.cu` 与 `V51` 同级**：V7 在 8192³ 下 26.909 TFLOPS，对 `V5` 同尺寸的 26.350 只有小幅领先。说明"降 tile + 提 occupancy"（33% → 50%）在大尺寸下收益有限 —— 又一次印证瓶颈不在 occupancy。
7. **`base.cu` 现在可以直接比了**：5070 Ti 上 base 已按 2048³ 重测（5.772 ms / 2.976 TFLOPS），所以"base 是 1024³ 不能比"这条限制只对 2080 Ti 的历史数据成立。

一句话总结：同一套 GEMM 代码，从 2080 Ti 到 5070 Ti 拿到约 2.4–3.2x（约 2.6x 中位），几乎是 FP32 峰值之比（~3.3x）的量级、远高于带宽比（1.45x）—— 说明该实现是**片上受限**而非显存受限；并且随着瓶颈从"访存宽度"转移到"shared bank / 延迟隐藏"， `float4` -类改动在新卡上收益锐减，后半程要靠 padding、Z-order 和流水线。

第二部分：自我检验清单详解

1. 分块后算术强度为什么从 0.25 提升到 32 FLOPs/Byte

算术强度 (Arithmetic Intensity, AI) 定义：

```
AI = 浮点运算量 FLOPs / 从 Global Memory 搬运的字节数 Bytes
```

它衡量「每搬 1 字节数据，能换多少次浮点计算」。AI 越大越偏向 compute-bound，AI 越小越偏向 memory-bound。

朴素实现（`base.cu`）：每个线程算一个 `C[row][col]`，沿 K 做内积。

- 每做 1 次乘加 MAC = **2 FLOPs**；
- 要从 Global 读 A 的一个 float（4 B）+ B 的一个 float（4 B）= **8 Bytes**。

```
AI = 2 FLOPs / 8 Bytes = 0.25 FLOPs/Byte
```

所以 base 版本几乎完全被显存带宽拖住（实测 1024^3 只有 0.911 TFLOPS）。

Block 分块（`V1.cu`，BM=BN=128，沿 K 以 BK 分段）：
一个 block 负责 C 的 `BM×BN` 分块，K 方向循环 `K/BK` 次；每个 k-tile：

- 计算量 = `2 · BM · BN · BK` FLOPs（每个输出元素累加 BK 次 MAC）；
- 全局访存 = `(BM·BK + BK·BN) · 4` Bytes（A、B 各搬一次，tile 内被所有输出复用）。

```
AI_tile = 2·BM·BN·BK / (4·BK·(BM+BN))
        = BM·BN / (2·(BM+BN))
```

代入 `BM = BN = 128`：

```
AI_tile = 128·128 / (2·(128+128)) = 16384 / 512 = 32 FLOPs/Byte
```

补充几点：

- **BK 被约掉了**，所以理想 tile 算术强度只由分块形状 BM、BN 决定，与 BK 无关；
- 这是**上界**：它假设 tile 数据在 block 内被完全复用、且只统计 Global 流量；
- A100 的 ridge point ≈ `19.5 TFLOPS / 1.55 TB/s ≈ 12.6 FLOPs/Byte`。32 已经跨过这条线，说明 128×128 的分块把 GEMM 从 **memory-bound 拉到了 compute-bound**，剩下的任务才是把算力（FFMA）真正喂满。

从整体流量看也一致：整张 C 需要把 A 读 `N/BN` 遍、B 读 `M/BM` 遍，
`AI_total = 2MNK / (4·(MK·N/BN + NK·M/BM))`，在 M=N=K、BM=BN 时同样回到 `BM·BN/(2(BM+BN))`。

2. 朴素实现 / Shared Memory 分块 / 线程分块 各自的瓶颈

三层优化的本质是把「复用」一层层下沉：Global → Shared → Register。瓶颈也随之迁移。

实现	复用在哪	主要瓶颈	ncu 表征
朴素 `base.cu`	无	**Global Memory 带宽 + 延迟**。每个 MAC 要 2 次 GMEM load，AI=0.25，访存指令数爆炸	`Stall Long Scoreboard` （等 L2/DRAM），Memory Throughput 接近峰值
Shared 分块 `V1.cu`	Block 内	**Shared Memory 带宽 / LSU 吞吐**。内层每 1 次 FMA 要 2 条 LDS（读 As、Bs），LDS:FMA = 2:1，很快打满 shared 端口	`MIO Throttle` 、 `Stall Short Scoreboard` ，可能伴随 bank conflict
线程/ 寄存器分块 `V2+` （外积）	线程寄存器内	**寄存器用量 → occupancy**，指令发射，以及**访存延迟隐藏**	`Stall Wait` 、occupancy 受限、寄存器溢出引起的 `Stall Long/Short Scoreboard`

关键点：

- 朴素版不是"算得慢"，是**搬得慢**——数据被重复搬运，带宽全浪费在冗余 load 上；
- Block 分块把 Global 流量降下来后，**shared 带宽**成为新天花板，所以后面必须做外积、float4、swizzle 来减少 LDS 条数与冲突；
- 线程分块把 shared 读数摊薄到 FMA 上，但每线程寄存器累加器 `TM·TN` 变大，**occupancy 下滑**，最终瓶颈变成"能不能用足够多的在飞访存 + 流水线把延迟盖住"。

3. 独立实现一个 Shared Memory Tiling + Thread Tiling 的 SGEMM

本仓库 `V1.cu` （block tiling）+ `V2.cu` （thread tiling）就是这条路线，结构骨架如下：

```cpp
constexpr int BM=128, BN=128, BK=8;   // block tile 与 K 步长
constexpr int TM=8,  TN=8;            // 每线程持有的输出小块
__global__ void sgemm(float* A, float* B, float* C, int M,int N,int K) {
    __shared__ float As[BM][BK];
    __shared__ float Bs[BK][BN];

    int row0 = blockIdx.y*BM, col0 = blockIdx.x*BN;
    // 每个线程在 block tile 内的 2D 坐标
    int tx = threadIdx.x % 16, ty = threadIdx.x / 16;

    float acc[TM][TN] = {0.f};

    for (int k = 0; k < K; k += BK) {
        // (1) 协作加载 (cooperative load)：256 线程平铺搬 A(BM×BK)、B(BK×BN)
        for (int i = threadIdx.x; i < BM*BK; i += blockDim.x)
            As[i/BK][i%BK] = A[(row0 + i/BK)*K + (k + i%BK)];
        for (int i = threadIdx.x; i < BK*BN; i += blockDim.x)
            Bs[i/BN][i%BN] = B[(k + i/BN)*N + (col0 + i%BN)];
        __syncthreads();                        // (2) 等整块 shared 就绪

        // (3) 外积：每个 k 步读 (TM+TN) 个 shared 值，做 TM*TN 次 FMA
        #pragma unroll
        for (int p = 0; p < BK; ++p) {
            float a[TM], b[TN];
            #pragma unroll
            for (int i = 0; i < TM; ++i) a[i] = As[ty*TM + i][p];
            #pragma unroll
            for (int j = 0; j < TN; ++j) b[j] = Bs[p][tx*TN + j];
            #pragma unroll
            for (int i = 0; i < TM; ++i)
                #pragma unroll
                for (int j = 0; j < TN; ++j)
                    acc[i][j] += a[i] * b[j];
        }
        __syncthreads();                        // (4) 防止覆盖未读完的 shared
    }

    // (5) 写回 C
    for (int i = 0; i < TM; ++i)
        for (int j = 0; j < TN; ++j)
            C[(row0 + ty*TM + i)*N + (col0 + tx*TN + j)] = acc[i][j];
}
```

要点：

1. **Block 分块**决定 Global 复用（AI 从 0.25 → 32）；
2. **Thread 分块（寄存器累加器 `TM×TN`）**决定 shared 复用（见第 4 条），并把累加结果留在寄存器，避免每步写 shared；
3. shared 加载用 `+4` padding 或 swizzle 消 bank conflict（见第 7 条）；
4. 用 `__launch_bounds__` 控寄存器（见第 9 条）；双缓冲/pipeline 见第 5、10 条；
5. 一定要 `Check = PASS`（本目录 V1–V5 头部都标了 PASS）。

4. 外积 (Outer Product) 为什么比内积提高 Shared Memory 复用率

内积视角（教科书定义）：`C[i][j] = Σ_k A[i][k]·B[k][j]`。
如果每算一个输出就沿 k 走一遍，每步读 1 个 A、1 个 B，做 1 次 FMA：

```
LDS : FMA = 2 : 1     → 每 1 次 FMA 就要 2 条 shared load
```

shared 带宽比 FMA 稀缺得多，于是 LSU/MIO 先被打满（第 2 条的瓶颈）。

外积视角：固定一个 k，取

- A 的一列 `a[0..TM-1] = As[k][.]`（TM 个值）
- B 的一行 `b[0..TN-1] = Bs[k][.]`（TN 个值）

然后做一次 **rank-1 更新** `acc[i][j] += a[i] * b[j]`，共 `TM·TN` 次 FMA：

```
LDS : FMA = (TM + TN) : (TM·TN)
```

取 `TM = TN = 8`：

```
LDS : FMA = 16 : 64 = 1 : 4
```

也就是**每 4 次 FMA 只需 1 条 shared load**，相比内积的 2:1 提升了约 **8 倍**的 shared 复用率。
本质：`a[i]` 沿 TN 方向被复用，`b[j]` 沿 TM 方向被复用，两个操作数都从寄存器里被"扇出"给多次乘加。

额外好处（配合 warp 形状，见第 8 条）：warp 内 8 个 lane 若共享同一个 `a[i]` 地址，shared 可以直接 **broadcast**，只占 1 次 wavefront，进一步省 shared 事务。

5. 双缓冲流水线中 LDG / FFMA / STS 的时序重叠

以 `V5.cu` （Turing，无 cp.async）的软件双缓冲为例。每轮迭代做三件事：

```
迭代 t:   |--- ① LDG(t+1): 把下一个 tile 读进寄存器 a_reg/b_reg，只发不等 ---|
          |============ ② FFMA(t): 用 shared[cur] 做一大段外积 ============|
                                                        |③ STS(t+1)| sync |
迭代 t+1:                                                              |=== FFMA(t+1) ===|...
```

时序关系：

- **LDG(t+1) 在迭代 t 开头就发出**，是非阻塞的；
- 它要等一整段 **FFMA(t) 外积计算**跑完才会被消费 → **访存延迟被计算时间盖住**（memory latency hidden by compute）；
- 为什么必须等 FFMA(t) 跑完才 STS：寄存器 `a_reg/b_reg` 里正存着 tile t+1 的数据，而 FFMA(t) 还在用 shared 里的 tile t；等算完再 `STS` 写进另一块 buffer `cur^1` ，避免覆盖正在读的那一块；
- 最后 `__syncthreads()` ：保证所有线程都写完 `cur^1` ，下一轮大家才能安全读；
- 于是形成「**算当前块 = 搬下一块**」的重叠，LDG 的延迟正好落在 FFMA 那段时间里。

关键约束：

- 只有 **1 个 tile 在飞**（单级 prefetch），寄存器里要一直压着 `a_reg/b_reg` ，所以 V5 必须 `__launch_bounds__(256, 2)` 把寄存器压住，否则 occupancy 掉到 25%；
- 想容错更深的延迟，就要**更深的流水线**——这正是 V6 用 `cp.async` + `STAGES=4` 做的事（见第 10 条）。

`V6.cu` 里没有独立的 LDG/STS，改用 `cp.async` 直接 Global→Shared，并用 `cp.async.wait_group<STAGES-2>` 作为"数据到齐"的栅栏，但**仍需接一个 `__syncthreads()`**：`wait_group` 只保证"当前线程自己发的拷贝"完成，其他线程填的 shared 得靠 block 同步才能读。

6. 用 Nsight Compute 识别 Long Scoreboard vs MIO Throttle

两个都出现在 `smsp__pcsamp_warps_issue_stalled_*` / Warp State Statistics 里，含义完全不同：

Stall 原因	含义	典型触发	你该做什么
Stall Long Scoreboard	warp 在等一个**长延迟**数据依赖，通常是 **global/local memory** load 的结果（L1TEX miss，要等 L2/DRAM）	朴素 GEMM、没做预取/流水线、寄存器溢出到 local	提高访存重叠：多级预取、双缓冲、 `cp.async` 、增加 occupancy/ILP、减少寄存器溢出
Stall MIO Throttle	MIO（Memory Input/Output）指令**队列满**：shared memory / SFU / 其他 MIO 类指令发得太密，来不及处理	shared load 过多（LDS:FMA 高）、bank conflict、没用 float4	**降 shared 指令数**：外积、float4 向量化 LDS/STS、swizzle/padding 消冲突、寄存器分块
（对照）Stall Short Scoreboard	等 **shared/local** 这种短延迟数据依赖	外积依赖链、bank conflict	提高 ILP、错开访问模式
（对照）Stall Barrier	卡在 `__syncthreads()`	线程负载不均、同步太频繁	双缓冲减少同步次数、均衡 work

一句话区分：

- **Long Scoreboard = 在等远处的 Global 数据回来** → 延迟没盖住；
- **MIO Throttle = shared/特殊指令发太多、队列堵了** → shared 吞吐不够。

实操顺序：先看 `Memory Workload Analysis` 判断是 DRAM 还是 Shared 打满，再看 Warp State 是 Long Scoreboard（→延迟/预取问题）还是 MIO Throttle（→shared 指令/冲突问题），对着改。

7. Shared Memory Bank Conflict 产生条件 + 3 种解决方案

硬件模型：shared memory 被划成 **32 个 bank**，每个 bank 宽 4 Bytes。
地址 `addr` 的 float 落在 `bank = (addr / 4) % 32` 。
一个 warp 的 32 个线程发出一次 shared 访问：如果落在**同一个 bank 但地址不同**，硬件无法一次服务，就会**拆成多笔事务**（n-way conflict），最坏 32-way，吞吐掉到 1/32。

产生条件：warp 内线程的访问地址 **跨 bank 的步长是 32 的倍数**（bank 编号重复），典型是**按列访问一个行主序的 shared 数组**：

```
As[row][k]  行主序，一行 128 个 float(=128 bank 位 → 128 % 32 = 0)
线程 t 读 As[0..31][k] → 地址相差 128 float → 全部 128 % 32 = 0
→ 32 条线程全部撞同一个 bank → 32-way conflict
```

3 种解决方案：

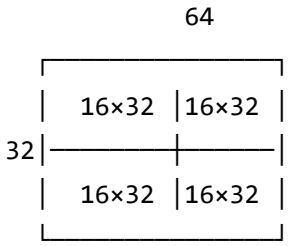
1. **Padding（补位）**：给行宽 +1 / +2 / +4，破坏"行宽是 32 倍数"这个条件。
   `As[BK][BM + 4]` （本仓库 V4Z/V6 的写法）： `+4` 既把每行的 bank 起点错开，又保持 16B 对齐，方便 `float4` 访问。
   经典 `+1` 做法是把每行银行号整体后移一位，使同一列相邻行的 bank 不同。
2. **Swizzle（异或置换）**：不改变数组大小，而是把逻辑下标映射成 `col' = col ^ (row * K)` （CUTLASS 风格），让每行的列索引按行号 XOR 旋转，物理 bank 被打散。
3. **改变访问模式 / 向量化**：让 warp 一次读**一整行**（连续地址，天然无冲突），或用 `float4` 一次读 4 个连续 float（每线程占 128 bit = 4 个 bank），配合外积的 broadcast，把多次标量访问合并成少数几笔事务。

判定口诀：**同一 bank、不同地址 → 冲突；同地址 → broadcast，不冲突；不同 bank → 并行，不冲突。**

8. 为什么 4×8 的 Warp 形状优于 1×32

把 32 个 lane 铺到 warp tile 上时，"1×32" = 一维条带，"4×8" = 二维网格（本仓库 `V4Z.cu` / `V6.cu` 用的是 4×8，每个 lane 再负责一个 2D 子块）。

以 V4Z 为例：warp tile 为 `32×64` ，lane 排成 4×8，每个 lane 持有 8×8（V4Z 里是 4 个 4×4）子块：



lane 布局 4×8，每 lane 一个 2D 子块

4×8 的优势：

1. **两个操作数都能复用（均衡的负载比）**：lane 是二维的，每线程自己就有 2D 寄存器小块，外积时 A 片段沿 N 复用、B 片段沿 M 复用， `LDS:FMA = (TM+TN):(TM·TN)` 很小；1×32 只能让每线程拿一维条带，一个方向没有复用，load 数暴增。
2. **匹配 broadcast/multicast**：4×8 中，同一 `lane_row` 的 8 个 lane 读同一个 A 地址 → 8 路 broadcast；同一 `lane_col` 的 4 个 lane 读同一个 B 地址 → 4 路 broadcast。shared 用**极少数 wavefront** 就服务完整 warp，银行压力最小。
3. **block tile 是二维的**： `BM×BN` 的 warp 划分天然是 2D（如 4×2 个 warp），二维 lane 形状才能和 warp tile 对齐、避免碎片。
4. **输出写回/访存更友好**：每个 lane 的 2D 子块写回 C 时地址更连续，便于 float4 与合并访问。

一句话：**1×32 把 warp 压成一维条带，等于放弃了其中一个维度的寄存器复用，让 shared load 和事务数上升；4×8 保持二维形状，A/B 两侧同时复用并且落在 broadcast 路径上。**

9. 给定 TM/TN/BM/BN 计算寄存器用量与 occupancy

每线程寄存器估算：

```
regs/thread ≈ TM·TN (累加器 c_frag)
            + TM   (a_frag)
            + TN   (b_frag)
            + 地址/索引/循环计数/临时量 (~20~40)
```

例如 `TM=TN=8` ： `64 + 8 + 8 = 80` ，加杂项通常报到 **128~168** 个寄存器/线程。

占用率 (Occupancy) 计算步骤：

1. 寄存器按 **warp 粒度 256 个**、**每线程 8 个**的倍数分配（向上取整）；
2. `每 SM 常驻 warp 数 = ⌊65536 / (regs向上取整 · 32)⌋` （A100 每 SM 有 64K 寄存器）；

3. `occupancy = 常驻 warp 数 / 硬件上限 warp 数` （A100 上限 64 warp = 2048 线程/SM）；
4. 同时受 **shared memory** 和 **每 SM 最大 block 数** 限制，取三者最小值。

代入本仓库参数（BM=BN=128, 256 线程, A100）：

- `V4Z.cu` 用 `__launch_bounds__(256, 2)` ，强制每个 block 至少 2 个常驻 → 编译器把寄存器压到 **≤128**；
- 常驻函数线程 = `65536 / 128 = 512` = **16 warp / SM**；
- occupancy = `16 / 64 = 25%` ；
- 共享内存： `As = BK·(BM+4)·4 = 4224 B` ， `Bs = BK·BN·4 = 4096 B` ，约 **8.3 KB/block**，2 个 block 仅 16.6 KB，不是瓶颈。

另一个方向：若去掉 `__launch_bounds__` ， `BK` 调大后寄存器会飙到 ~141，则 `65536/(141·32)` 取整后只能驻留 1 个 block（8 warp） → occupancy 掉到 **25% 以下（实际 12.5%）**，还会出现寄存器溢出到 local memory，ncu 里表现为 Long Scoreboard。

结论： `TM·TN` 越大，寄存器越紧、occupancy 越低，但 shared 复用越高——这就是 queue 里 `TM/TN/BK/padding` 需要调参的根本原因：**在"喂饱 FMA"和"保住 occupancy"之间找平衡。**

10. cp.async 相比传统 LDG + STS 的优势

传统路径（ `V5.cu` ）： `Global --LDG--> 寄存器 --STS--> Shared` ，两步走。 `V6.cu` 用 Ampere 的 `cp.async` （LDGSTS）直接 `Global --> Shared` ，不经过寄存器。

优势：

1. **省寄存器、提 occupancy**：不再需要 `a_reg/b_reg` 中转寄存器，释放出的寄存器可以给累加器或让更多 block 常驻（V5 要靠 `__launch_bounds__` 硬压，V6 压力骤降）。
2. **省指令、降发射压力**：一条 `cp.async` 取代 "LDG + STS" 两条，指令数更少，smsp 发射更轻松。
3. **真正异步、可多级在飞**： `commit_group` / `wait_group` 机制允许同时有 `STAGES-1` 个 tile 在路上，延迟容错从 V5 的"1 个 tile"提升到 **多级深度流水线**，能盖住更长的 DRAM 延迟。
4. **可绕过 L1**： `.cg` 变体只走 L2，不污染 L1、不占用寄存器，适合 GEMM 这种一次性大块流式数据。
5. **和计算重叠更彻底**：硬件在后台把数据搬进 shared，warp 只管发指令，继续对上一块做 FFMA。

注意 / 坑（V6 注释里也写了）：

- 只支持 **sm_80+**；sm_75 会退化成同步拷贝，没有异步收益；

- `cp.async` 一次只能搬**连续**的 4/8/16 B：B 无需转置可 16B（`.cg`），A 转置后 16B 块被打散，只能用 4B（`.ca`）；
- `wait_group` **只保证当前线程自己发的拷贝完成**，等完必须再接 `__syncthreads()` 才能读整块 shared；
- 尾部不足 `STAGES-1` 个 tile 时也要 `commit` 一个**空组**，否则"已提交组数"变少会让 `wait_group<STAGES-2>` 推出错误结论、读到还在搬的数据。

附：清单 ↔ 本仓库版本对照

清单条目	对应版本 / 文件
算术强度推导	`base.cu` (0.25) → `V1.cu` (32)
三种瓶颈	`base.cu` / `V1.cu` / `V2.cu`
Shared+Thread Tiling	`V1.cu` 、 `V2.cu`
外积复用	`V2.cu` 起
双缓冲流水线	`V5.cu` （软件）、 `V6.cu` （cp.async）
Long Scoreboard / MIO Throttle	用 Nsight Compute 现场 profile（原文的 `*.ncu-rep` 已不在目录中）
Bank conflict	`V4Z.cu` （padding + Z-order lane）
4×8 warp 形状	`V3.cu` 起， `V4Z.cu` / `V6.cu` 定型
寄存器 / occupancy	`__launch_bounds__(256,2)` ， `V4Z.cu` 注释
寄存器 / occupancy（继续降 tile）	`V7.cu` （8×4 tile、85 寄存器、3 block/SM）
cp.async 优势	`V6.cu`